

Lab 3: Architectural Pattern Selection — Written Justification

Student Project Portfolio & Showcase App

Architecture Selection

We chose Microservices Architecture for the Student Project Portfolio & Showcase App system.

Reason 1: Independent Scaling for Traffic Spikes

Public voting and the live leaderboard experience extreme, unpredictable traffic spikes during the expo (NFR-001 requires under 300ms latency even under peak load), while judging and admin operations remain low-traffic throughout. A microservices architecture lets us scale the Voting & Leaderboard Service horizontally and independently, without wastefully over-provisioning the Judging Service or Event Admin Service, which see comparatively little load.

Reason 2: Fault Isolation Across Independent Workflows

Project submission, judging, voting, and event administration are functionally independent workflows run by different actors (Student Team, Panel Judge, Public Voter, Event Admin) at different times. If the GitHub repository validation logic inside the Project Submission Service fails or is slow, it must not block live voting or leaderboard updates. Microservices isolate failures to a single service, so the rest of the expo continues to run uninterrupted — a Layered or Client-Server design would risk a single shared failure point across all these workflows.

Security Advantage

Each microservice enforces its own authorization boundary scoped to its actor type — for example, the Voting & Leaderboard Service independently enforces one-vote-per-verified-account checks (anti-sybil protection, FR-003) without depending on the internal logic of the Judging or Admin services. If one service were compromised, the blast radius is contained to that service's data and permissions rather than exposing the entire system, unlike a monolithic Layered architecture where a single breach can expose all business logic.

Performance Benefit

The Voting & Leaderboard Service can be deployed with its own caching layer (e.g., an in-memory store) and scaled on dedicated infrastructure to meet the sub-300ms real-time update requirement (NFR-001), independent of heavier, less time-sensitive operations like rubric-based judging or event configuration. This targeted performance tuning would not be possible if all functionality were bundled into a single deployable unit, as in a Layered architecture.